Collective Communication for Modern GPU Training in gem5 Garnet

An empirical study of multicast, express links, and tensor streaming

Chunyu Liu and Boyan Pu

September 6, 2026

Abstract

Distributed GPU training repeatedly pauses computation for communication. We study how collective structure can receive direct support in a Garnet Mesh network. The study covers three mechanisms—router-level tree multicast, physical express-link bypass, and multi-flit in-network all-reduce—and uses an executed eight-H100 collective trace as an integrated case study. The evaluation covers 162 multicast pairs, a 6,144-pair bypass screen plus a 1,536-pair refinement, and paired 30-request broadcast and 15-request tensor replays, backed by functional, backpressure, and cross-mechanism validation. Tree multicast cuts internal-link flits by 39.51% on average (26.19%, 39.22%, and 53.13% for 4, 8, and 16 destinations), but its atomic fanout path has ten throughput regressions. In the broad offline route-table topology sweep, source-only distance-scaled diagonal links provide a 1.131× geometric-mean latency speedup, while source-only stride links are neutral at 0.999×. Allowing cycle-aware, dimension-ordered multi-hop stride bypass with congestion admission raises the full-matrix result to 1.430× and throughput by 21.87%. Finally, preserving 15 multi-flit tensor requests versus serializing 6,720 scalar lanes reduces the scaled replay window from 80.638 to 17.055 million ticks (4.728×). All measurements are cycle-level Garnet results: the mechanism studies use controlled synthetic traffic, while the integrated case study replays the measured H100 workload.

1 Introduction

Modern training gives the network a repeated synchronization contract. NCCL exposes collectives such as broadcast and all-reduce across GPU ranks [3]. Each GPU computes locally, exchanges a gradient or activation, and waits until the collective result is available before the next step can consume it. The same contract produces three distinct network opportunities:

1. **Replication:** a broadcast can share a common route prefix and replicate only at branch Routers.
2. **Path length:** a unicast packet can skip intermediate Routers when a physical express link is worth its wire cost and congestion.
3. **Representation:** an all-reduce tensor can remain a streaming multi-flit request in place of thousands of serialized scalar operations.

This report focuses on Lab 4 Topic 3 (Microarchitecture), which requires both Bypass and Multicast. Tensor all-reduce and replay are an additional Topic 4 extension that makes the AI-training setting concrete. The implementation has three corresponding parts:

- **Router-level tree multicast** shares common route prefixes and replicates a single destination-bitmap packet at branch Routers.
- **Physical express-link bypass** adds shortcut links and uses an offline route table to select cycle-saving express hops; the refined stride design may use several monotonic hops and applies congestion-aware admission at every eligible Router.
- **Tensor all-reduce replay** keeps a measured collective as a multi-flit request, reduces lanes in Routers, and broadcasts the result.

Table 1: Main controlled-study configurations.

Parameter	Multicast matrix	Bypass screen
Topology	4×4 Mesh, source Router 5	4×4 and 8×8 Mesh
Traffic	4/8/16 destinations; optional uniform background	uniform, transpose, bit complement, hotspot
Packet size	1/4/16 flits	1/4/16/64 flits
Load points	0.1/0.5/1.0, background 0/0.1	16 points from 0.01 to 3.0
Seeds	1, 7, 17	1, 7, 17
Measurement	20 warm-up, 100 measured, 20 cooldown requests	1,000 warm-up, 5,000 measurement, 100,000 drain cycles

The primary performance experiment is the offline route-table bypass screen across topologies and wire models. The multicast matrix provides the matching mechanism-level evidence, while the H100 replay supplies a concrete trace case study alongside the factorial evaluation.

The mechanisms show different benefit profiles. Multicast has a robust traffic benefit and a workload-dependent throughput benefit. Bypass gains depend on topology, wire cost, and load; structured or loaded traffic benefits when admission selects a useful shortcut. Tensor streaming changes the logical request representation while preserving rank contributions and Router flit counts. Each result is reported in its native unit: traffic reduction, latency ratio, or trace-window ratio.

Section 2 defines the mechanisms, baselines, and metrics; Section 3 maps them to Garnet components and records the correctness contract. Section 4 evaluates multicast and express-link bypass under controlled synthetic traffic, and Section 5 replays the measured H100 collective trace. The final sections record interpretation limits, authorship, and reproducibility.

2 Mechanisms and experimental setup

The implementation exposes two Topic 3 mechanisms and one Topic 4 extension. The baseline and metric definitions below fix the comparison contract before the mechanism descriptions and quantitative results.

2.1 Experimental setup: variants and baselines

The implementation uses gem5 v23.0.0.1 [1] (base revision `77fcf26d57`) and deterministic XY routing on the cycle-level Garnet network model [2]. The source-only bypass screen was rerun at project revision `135768001a97`; the multi-hop refinement is based on `1e8764cde7`. Both manifests record hashes for every simulator source, including uncommitted refinement files. A reported ratio always compares the same topology, traffic, packet sequence, payload, request schedule, and seed. The multicast comparison uses replicated unicast as its baseline: the baseline injects one complete packet for each destination, while the proposed path injects one destination-bitmap packet and replicates it at branch Routers. Both variants use the same Mesh, and every selected destination must receive a complete packet. The bypass comparison uses plain Mesh XY as its baseline; the proposed Mesh_Bypass adds physical express links and an audited route table, while injection, workload, and ordered-VNet behavior remain unchanged.

The 4×4 diagrams below are illustrative mechanism examples, not the full topology scope. The main bypass screen covers both 4×4 and 8×8 meshes, the H100 replay uses a 2×4 mesh, and the multicast performance matrix uses the 4×4 setup shown in Figure 1.

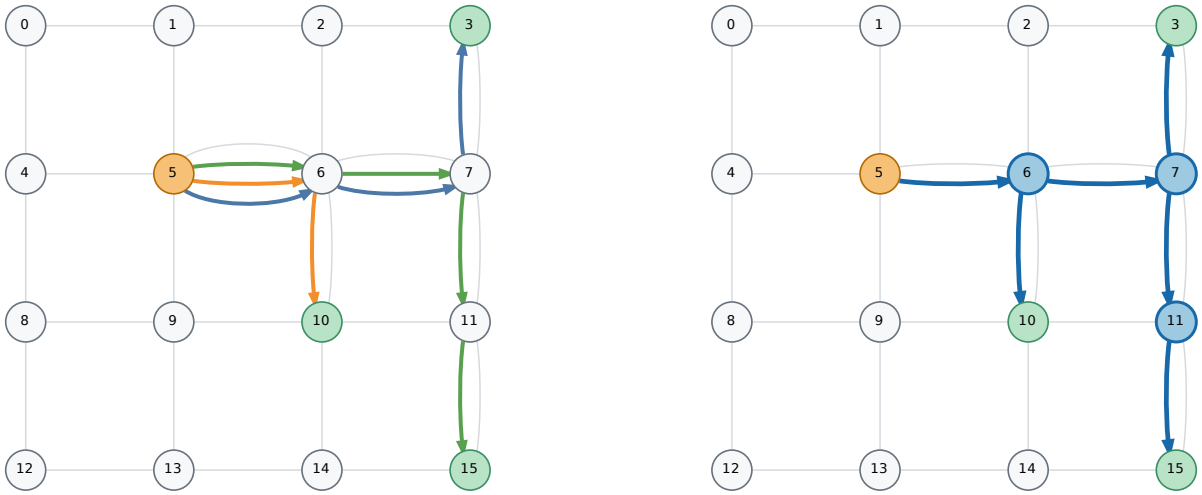
Figure 1 visualizes the multicast comparison before the metric definitions below.

For a paired latency L, internal-link flit count F, and throughput Q, we use

$$S_L = L_{\text{base}}/L_{\text{new}}, \qquad R_F = 1 - F_{\text{new}}/F_{\text{base}}, \qquad I_Q = Q_{\text{new}}/Q_{\text{base}} - 1.$$

Thus $S_L > 1$ is faster, $R_F > 0$ is less link traffic, and $I_Q < 0$ is a throughput regression. One multicast request completes only after every selected destination receives the complete packet; Q counts these logical requests per cycle, so both variants perform the same application-level work. We use arithmetic means for additive changes and geometric means for latency ratios. The latter is essential for bypass because near-saturation ratios have a long positive tail. The released CSV files retain full distributions, extrema, and regression cases.

Table 1 collects the parameters that otherwise tend to be hidden in command lines. Each comparison is paired by workload and seed.



(a) Replicated-unicast baseline **(b) Proposed tree multicast**

Figure 1: Illustrative 4×4 multicast baseline and proposed path.

2.2 Tree multicast

The baseline Network Interface injects one physical packet per destination. The multicast path injects one packet carrying a 64-bit destination bitmap. Each Router removes destinations already served, computes the required output branches, and makes a local copy only where selected. Head, body, and tail state is retained per branch. Fanout is atomic: a flit advances only when all selected output VCs have credit. This preserves exact multi-flit delivery, but it also couples a fast branch to a blocked branch—an important interpretation of the throughput results below. The bitmap bounds one collective domain to 64 Routers.

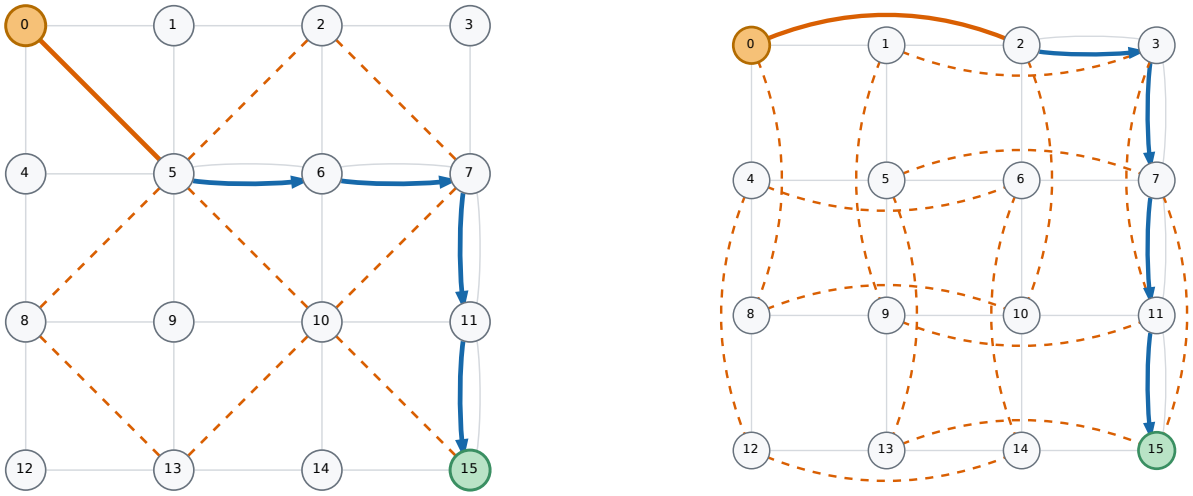
2.3 Offline route-table express-link bypass

`Mesh_Bypass` appends bidirectional diagonal or stride links to the ordinary Mesh. Before simulation, a route-table builder enumerates every current-Router/destination pair. The original screen permits one source express hop followed by XY. The refinement lets dimension-aligned stride links be selected repeatedly in the X phase and then the Y phase, but only when the complete suffix has lower modeled cycle cost. The builder constructs the full paths and channel-dependency graph and rejects any placement whose graph is cyclic.

- **Optimistic:** every express link costs one cycle, an upper bound on the value of shortening a path.
- **Distance-scaled:** latency and serialization scale with the Manhattan span, a more conservative proxy for a physical wire.

The cost of an ordinary edge is link latency plus Router latency. An express edge uses its configured wire latency plus one Router traversal and is admitted statically only if it lowers the full suffix cost. Ordered Vnets retain this deterministic table. On unordered traffic, the refined selector may fall back to the monotonic XY edge when the express output or landing output is blocked, the express output is materially more congested, or the destination is detected as a sustained hotspot.

1. Each Router consults the table entry indexed by its own ID and the destination.
2. A stride express entry must move monotonically toward the destination in the current XY dimension and may not overshoot.
3. Both adaptive choices therefore remain in X-before-Y order.



(a) Diagonal placement **(b) Stride-2 placement**

Figure 2: Illustrative 4×4 bypass placements and route-table-selected paths.

Legend: orange dashed lines are installed bidirectional bypass links; solid orange is the selected express hop or hops; solid blue is the ordinary DOR segment; gray lines are ordinary Mesh links.

Table 2: Implementation map in gem5 Garnet.

Mechanism	Main components	Added state and behavior
Multicast	`flit`, NI, `Router`	64-bit destination mask, per-packet branch VCs, Router-side replication, exact completion tracking
Bypass	`Mesh_Bypass.py`, `bypass_oracle.py`, `RoutingUnit`	Physical express ports, all-pairs per-Router next-hop table, monotonic DOR paths, channel-dependency audit
Tensor	NI, `Router`, replay compiler	Per-flit lane ID, per-(collective, lane) accumulator, queued backpressure-safe reduce/broadcast forwarding

4. A full all-pairs channel-dependency DAG certifies the resulting order.

The bypass screen uses data VNet 2 (four buffers per VC). The express-link cost is either optimistic (one cycle) or distance-scaled by the link's Manhattan span; the latter is the hardware-relevant model used for the main comparison.

The bypass comparison therefore has a plain Mesh XY baseline and a Mesh_Bypass variant with physical shortcut candidates. Figure 2 shows the two candidate placements. The diagonal design retains the original source-only policy; the stride refinement can select multiple aligned express hops along the deterministic dimension-ordered path.

3 Implementation and validation

The implementation changes the packet metadata, Network Interface (NI), Router fast path, routing unit, and topology construction rather than emulating the mechanisms in the traffic generator. Table 2 summarizes the division of responsibility.

3.1 Atomic multicast fanout

The head flit first computes the pruned-tree children selected by its bitmap. The Router commits branch state only if every required output has a free VC; body and tail flits reuse those VCs. Listing 1 shows the essential

Listing 1: Core of Router-side atomic multicast branch allocation (`Router.cc`).

```
for (const auto& output : m_output_unit) {
    const auto& direction = output->get_direction();
    if (direction == "Local") continue;
    const int child = collectiveChildId(direction);
    if (multicastChildNeeded(mask, m_id, child))
        destinations.push_back(child);
}
for (const int destination : destinations) {
    const int outport = collectiveOutportForChild(destination);
    if (outport < 0 || !m_output_unit[outport]->has_free_vc(vnet))
        return false;                // allocate no partial fanout
}
for (const int destination : destinations) {
    const int outport = collectiveOutportForChild(destination);
    const int outvc = m_output_unit[outport]->select_free_vc(vnet);
    m_multicast_branches.push_back({outport, outvc, destination});
}
```

Listing 2: Cycle-aware multi-hop DOR selection and dependency audit (`bypass_oracle.py`).

```
def best_stride_step(current, destination):
    if current == destination:
        return 0, None
    ordinary_next = xy_step(current, destination)
    suffix, _ = best_stride_step(ordinary_next, destination)
    choices = [(ordinary_edge_cost + suffix, 0,
                ordinary_next, "", None)]
    for link in outgoing[current]:
        if not dor_express(link, destination):
            continue
        suffix, _ = best_stride_step(link["dst"], destination)
        cost = int(link["latency"]) + router_latency + suffix
        choices.append((cost, 1, link["dst"],
                        link["name"], link))
    cost, _, _, _, selected = min(choices)
    return cost, selected

while current != destination:
    selected = next_links[current * routers + destination]
    next_router = selected["dst"] if selected else xy_step(
        current, destination)
    channels.append(selected["name"] if selected else
                    f"XY_{current}_to_{next_router}")
    current = next_router
for channel, successor in zip(channels, channels[1:]):
    dependency_graph[channel].add(successor)
if len(topological_order) != len(dependency_graph):
    raise ValueError("channel dependency graph is cyclic")
```

two-phase allocation. This makes a multi-flit packet exact and prevents partial branch allocation, but also explains the measured head-of-line coupling: one blocked branch stalls the whole fanout.

3.2 Auditable bypass routing

Topology construction creates two directed Garnet links for every installed physical shortcut and scales their latency by Manhattan span in the distance-scaled model. Listing 2 shows the refined stride policy: dynamic programming compares the full ordinary and express suffix costs, while the admissibility predicate preserves dimension order. The same full paths feed the dependency audit.

3.3 Streaming tensor reduction

Tensor packets retain their multi-flit boundary while each flit carries a lane identifier. Listing 3 is the Router accumulation point. An entry is released as soon as every local/child contribution arrives; completed lanes wait in a FIFO when their next output VC is unavailable, so backpressure does not discard reduction state.

Listing 3: Condensed lane-wise Router accumulation for tensor all-reduce (`Router.cc`).

```
const int lane = t_flit->get_lane_id();
const auto key = std::make_pair(t_flit->get_collective_id(), lane);
CollectiveLaneState& state = m_collective_lanes[key];
state.sum += t_flit->get_value();
++state.count;
if (state.count == m_collective_expected_fanin) {
    const int64_t sum = state.sum;
    m_collective_lanes.erase(key);
    if (m_collective_root)
        forwardCollectiveLane(sum, lane, CollectiveOp::Broadcast,
                              m_id, t_flit);
    else
        // parent is derived from m_collective_parent_outport
        forwardCollectiveLane(sum, lane, CollectiveOp::Reduce,
                              parent, t_flit);
}
```

Table 3: Executed validation coverage in the accepted snapshot.

Gate	Coverage	Required invariant
Multicast matrix	208 paired cases	Exact bitmap, flit order, no missing/duplicate delivery
Bypass screen/refinement	10,752 runs	Complete table, route termination, acyclic CDG
Multicast × bypass	624 runs	416 paired interactions complete without route corruption
Tensor functional	14 cases	Exact lane count and reduced value on varied meshes
Tensor backpressure	32 cases	Completion under varied link/router latency and occupancy
Trace tools	12 unit tests	Schema, sizes, ordering, IDs, and participant validation

3.4 Validation contract

Correctness gates check payload values and delivery sets, not only simulator termination. The compact-data builder additionally rejects incomplete pairs, non-finite metrics, inconsistent route metadata, and wrong run counts before generating any headline figure.

4 Evaluation results

4.1 Multicast performance

Tree multicast removes physical link work reliably, but its throughput benefit depends on branch synchronization. The results below separate those two claims rather than treating traffic reduction as a proxy for throughput.

Across 162 paired cases, tree multicast reduces internal-link flits by 39.51% on average (median 41.18%, range 16.67–53.13%). The effect is monotone in fanout: 26.19% for four destinations, 39.22% for eight, and 53.13% for sixteen. This is the cleanest evidence for the mechanism because both variants use the same Mesh and the metric counts only physical link flits. Figure 3 summarizes the reduction by fanout.

Latency also improves in this matrix: geometric-mean speedup is 3.638× (median 3.588×, range 1.200–22.512×), and all 162 paired values exceed one. The multicast implementation uses a dedicated Router fast path, so its switch-allocation and crossbar sequence differs from unicast. We therefore read latency as an implemented-system result and use link flits for the structural comparison.

Lower link traffic does not always translate into higher throughput. The arithmetic mean change is +190.89%, but the distribution is highly skewed by full-group traffic; ten of 162 cases regress, all at four destinations, with a worst case of −18.24%. The paired distribution is shown in Figure 4. Atomic fanout explains how both statements can be true: the tree does less total link work, yet a single blocked branch can hold up all copies.

4.2 Multicast sensitivity and tradeoff

The workload breakdown makes the dependency concrete. At 4, 8, and 16 destinations, mean throughput changes are +34.38%, +154.87%, and +383.42%, but the four-destination group contains all ten regressions. By packet size, the mean changes are +261.65% (1 flit), +169.45% (4 flits), and +141.57% (16 flits); the negative cases concentrate in the longer, loaded four-way requests. Figure 5 shows the same pattern together with the min–max spread, while Figure 6 relates traffic reduction to throughput on a case-by-case basis.

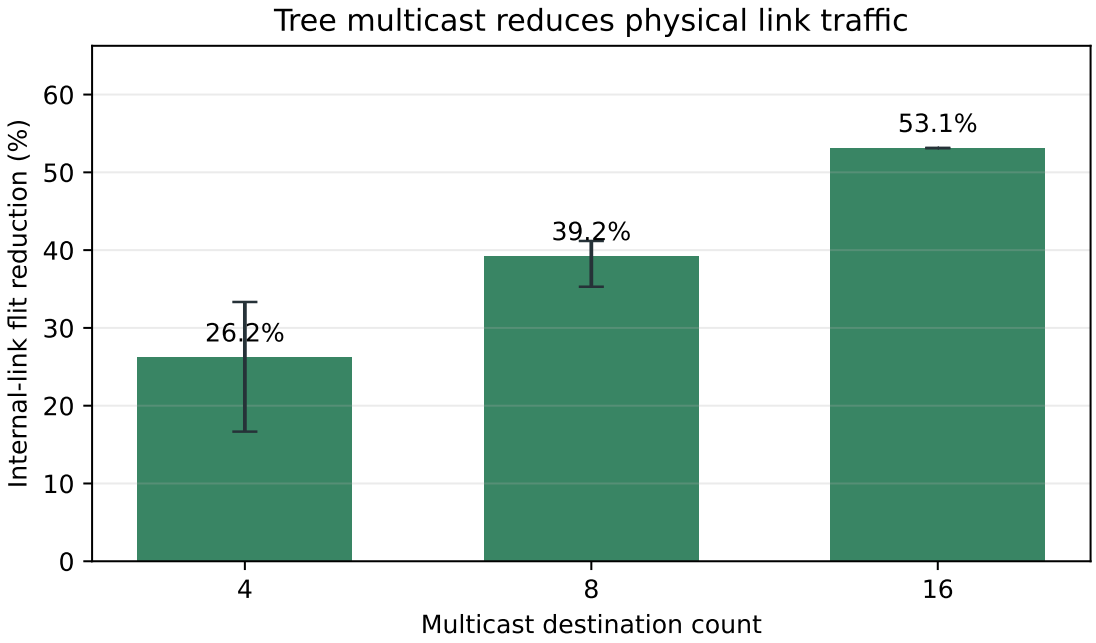


Figure 3: Internal-link flit reduction from tree replication; bars are means and whiskers show the observed range for each fanout.

Multicast takeaway. Tree replication reliably removes duplicate physical work, and the saving grows with fanout. Throughput follows a different workload curve because atomic fanout and the dedicated fast path introduce their own scheduling effects.

4.3 Primary experiment: offline route-table bypass

Express-link bypass is useful only when path shortening survives wire cost and contention. The screen below tests that claim across topology, traffic, and wire-model choices.

The bypass sweep uses four standard synthetic traffic patterns. In `uniform_random`, each source chooses a destination uniformly at random. In `transpose`, a source at mesh coordinate (x, y) sends to (y, x), creating a structured permutation. In `bit_complement`, it sends to the mirrored coordinate $(W-1-x, H-1-y)$, pairing opposite locations in the mesh. In `hotspot`, 50% of packets target one fixed Router (Router $N/2$ in an N-node mesh) and the rest use uniform random destinations. These four patterns isolate route structure and contention.

The original offline route-table sweep compares the same synthetic workloads and seeds on Mesh XY and on each physical express-link design, with the route table selecting at most one source express hop. The latency column in Table 4 is the geometric mean over all 1,536 paired seed cases per design; additive changes use arithmetic means. The fixed table isolates the value of an admissible shortcut, while the regression count exposes how often the average does not transfer to an individual workload. Stride removes more Router traversals, but its physical wire cost is higher and its latency benefit is smaller than diagonal links.

Table 4: Original source-only topology and wire-model screen over 1,536 paired seed cases per design; latency ratios use geometric means.

Design	Latency speedup	Regressions	Throughput change	Traversal change
Diagonal, distance-scaled	1.131×	350/1,536	+9.41%	+5.49%
Diagonal, optimistic	1.148×	164/1,536	+9.44%	+5.53%
Stride, distance-scaled	0.999×	539/1,536	+2.72%	−11.35%
Stride, optimistic	1.024×	321/1,536	+2.81%	−11.26%

Traversal change is $F_{\text{new}}/F_{\text{base}} - 1$ for each run's *aggregate* Router traversals. Positive throughput can therefore produce a positive traversal change even when individual paths shorten; the two columns must be read together.

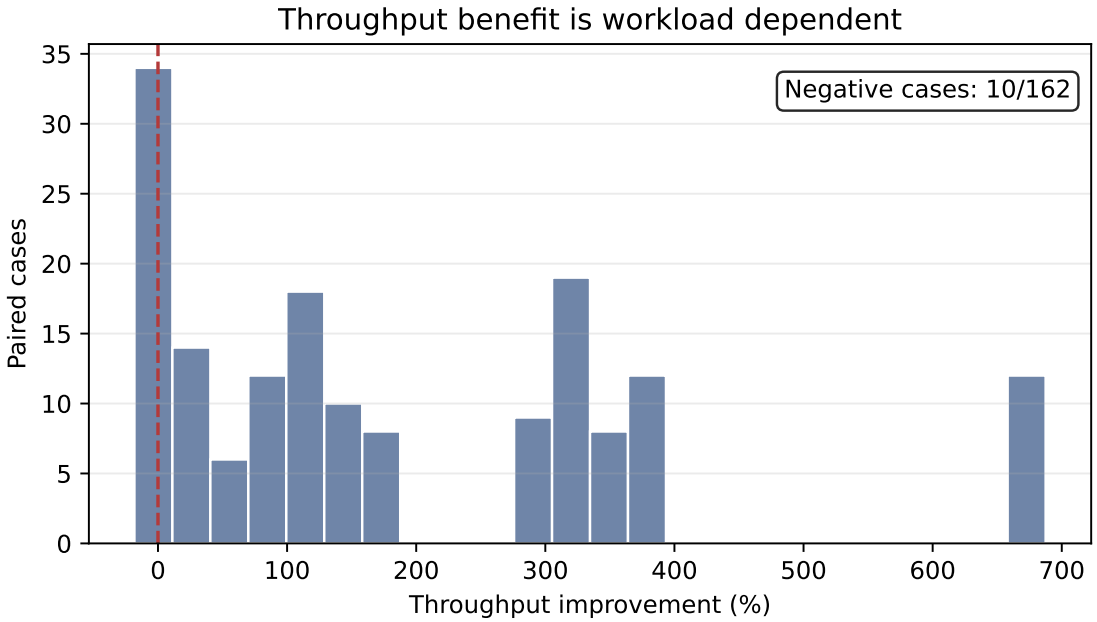


Figure 4: Paired multicast throughput changes, including the high-fanout tail and the ten regression cases.

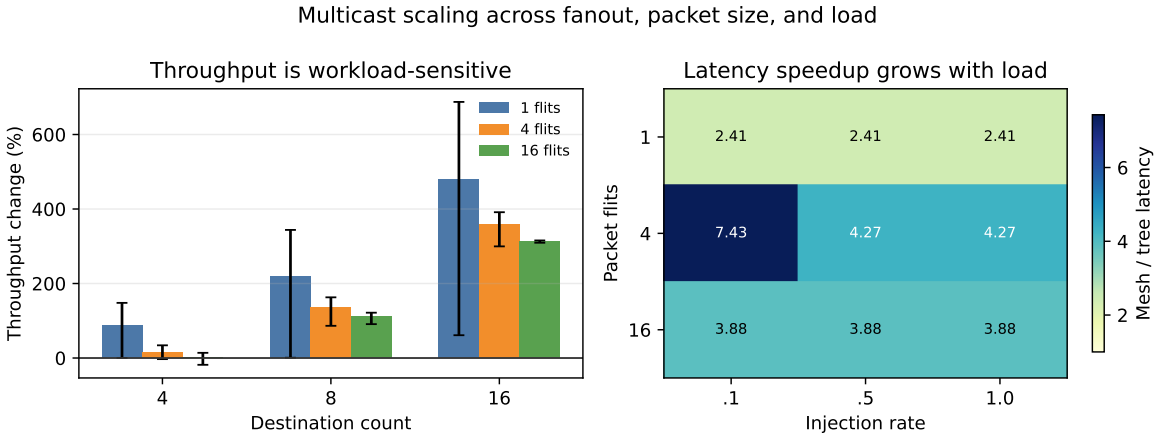


Figure 5: Sensitivity of multicast throughput and latency to fanout, packet size, and offered load.

The cost records explain the design choice. A 4×4 diagonal placement adds 9 undirected links (wire proxy 18, maximum radix 8), while stride adds 16 (wire proxy 32, maximum radix 6); on 8×8 the corresponding link counts are 49 and 96. Physical topology choice therefore combines hop reduction with wire length, radix, and buffer cost.

Figure 7 adds the multi-hop refinement to the four source-only designs.

In the source-only screen, diagonal links provide the strongest latency result, while stride links remove more Router traversals but incur a larger physical-link budget. At offered load ≤ 0.1, however, the distance-scaled geometric means are 0.999× for diagonal and 0.993× for stride. The gain therefore comes from favorable loaded cases rather than a universal reduction in base latency. The optimistic wire model is a sensitivity bound; distance-scaled latency is the hardware-relevant comparison.

4.4 Multi-hop stride refinement

The neutral source-only stride result identifies a structural limit: a packet can skip only one Router even when its path crosses several aligned shortcuts. The refined policy consults the cycle-aware table at every Router, so a packet may use several stride links while remaining monotonic in X and then Y. For unordered data traffic, conservative admission retains the static shortcut unless the express or landing output is blocked, the express output is materially

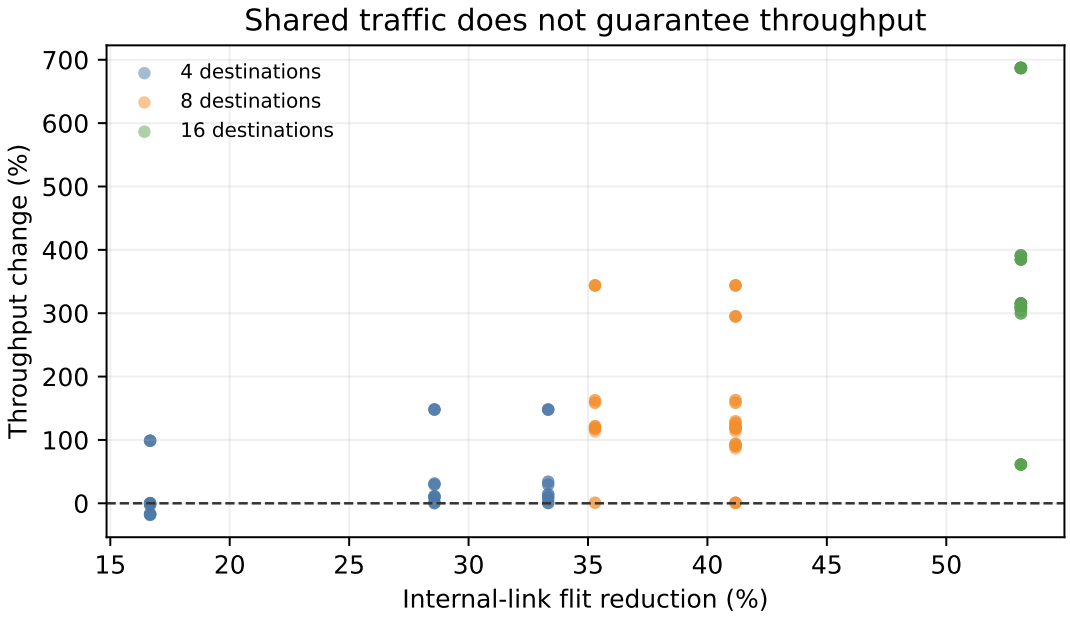


Figure 6: Traffic reduction versus throughput change for individual paired cases; the dashed line marks unchanged throughput.

more congested than XY, or epoch counters identify the destination as a sustained hotspot. For packets above 32 flits, the selector also checks credit headroom and the oracle-selected output at the landing Router, admitting the shortcut only when XY is under pressure and the landing output is idle. A 96-pair pilot rejected a more aggressive version despite its higher mean because it introduced four regressions larger than 5%; the retained guard had none in the same pilot.

Across all 1,536 pairs, the median is 1.082× and 243 ratios are below one, but only one regresses by more than 5%. The worst is 0.909× for 4×4, 16-flit hotspot traffic at offered load 0.3. At offered load at most 0.1 the geometric mean is 1.051×; above 0.7 it is 1.627×. The 64-flit group now reaches 1.134× with no regression larger than 5%. Thus the 1.430× headline is not a claim of universal base-latency improvement: multi-hop bypass is most valuable when the skipped Router stages also relieve loaded paths, while hotspot admission largely restores the baseline on many-to-one traffic.

Bypass takeaway. Express links are useful when route structure makes a saved hop valuable, while distance-scaled wires and extra radix impose real costs. Moving from one source-only shortcut to audited multi-hop DOR raises

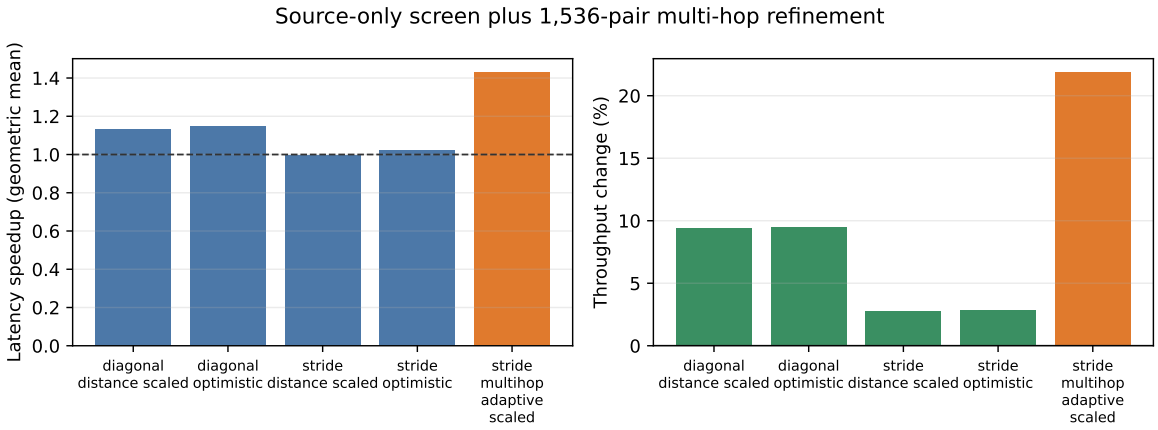


Figure 7: Latency speedup and throughput change for the source-only screen and the distance-scaled multi-hop adaptive refinement.

Table 5: Distance-scaled multi-hop stride refinement. Latency is a geometric mean; throughput and Router-traversal-per-delivered-flit changes are arithmetic means. The last column counts regressions larger than 5%.

Group	Pairs	Speedup	Throughput	Router/flit reduction	$> 5\%$ regress.
Overall	1,536	1.430×	+21.87%	18.10%	1
4×4	768	1.291×	+4.37%	17.37%	1
8×8	768	1.583×	+39.37%	18.84%	0
Uniform random	384	1.842×	+19.63%	24.98%	0
Transpose	384	1.182×	+5.50%	15.92%	0
Bit complement	384	1.892×	+61.07%	14.54%	0
Hotspot	384	1.015×	+1.28%	16.99%	1

Table 6: Multicast–bypass interaction over 104 pairs per row.

Multicast mode	Wire model	Link-flit reduction	Latency regressions
Replicated unicast	Distance-scaled	16.10%	39/104
Replicated unicast	Optimistic	16.10%	0/104
Tree multicast	Distance-scaled	8.33%	15/104
Tree multicast	Optimistic	8.33%	0/104

geometric-mean speedup from 0.999× to 1.430× for stride links; congestion admission reduces the severe-regression tail without weakening the wire model.

4.5 Multicast–bypass interaction

A 624-run factorial gate combines both multicast modes with Mesh XY and the bypass topology. All 416 Mesh/bypass pairs complete with exact destination sets and unchanged wire-distance accounting. Table 6 shows that shortcut use remains mechanism-dependent: replicated unicast can route each copy independently, whereas tree multicast preserves its pruned XY branch semantics. This gate establishes composability; it is not substituted for the broader unicast bypass screen above.

5 H100 trace case study

The H100 replay preserves the measured collective's ordering while exposing a request-representation effect: tensor streaming shortens the replay window without changing rank contributions or Router work.

This section pairs the factorial multicast and bypass evaluation with a trace-specific lowering, validation contract, and replay result. Its scaled tick counts remain in a separate trace workload and the main-study aggregates retain their factorial scope.

5.1 Trace lowering and implementation

The input is an executed eight-H100 80GB HBM3 NCCL microbenchmark (PyTorch 2.8.0+cu128, CUDA 12.8, NCCL 2.27.3). The replay compiler selects the 15 measurement-phase broadcasts and 15 all-reduces, represents communication with 16-byte flits, and scales payload bytes and relative release times by 1/1024. This preserves event ordering while making the trace practical for cycle-level replay.

For broadcast, each selected source event is lowered to variable-length tree-multicast packets. For all-reduce, the scalar replay creates independent serialized lanes; each lane converges rank values in the Routers and then travels back down the convergence tree. The tensor replay keeps each event as one multi-flit logical request, tracks a lane ID per flit, and performs the same Router-side reduction and tree broadcast while retaining the request's multi-flit boundary. The scalar and tensor paths are distinct collective implementations from the bitmap-based `tree_multicast` path; all three exploit shared tree prefixes.

Figure 8 connects the datapath to the replay representation. It also makes clear why the tensor comparison preserves the same contribution and Router-flit counts: only the logical request boundaries and completion scheduling change.

All three replay runners use the same offline route-table Mesh_Bypass arm as the main bypass study: diagonal links with a four-link budget and distance-scaled wire latency. Mesh XY is the matched baseline. In this H100

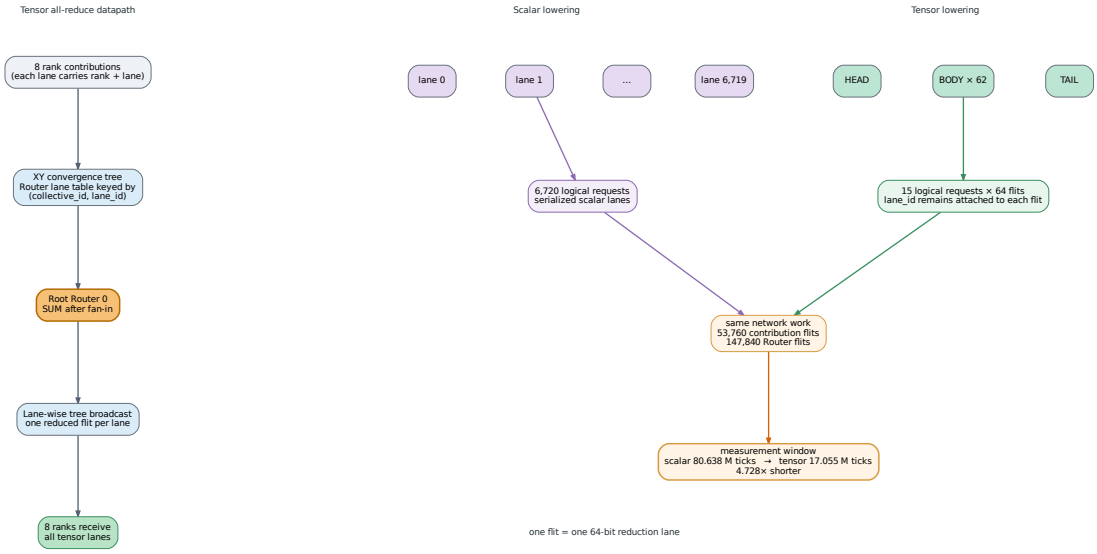


Figure 8: Tensor all-reduce datapath and the scalar-lane and tensor-request replay lowerings.

placement every collective path uses the ordinary XY route, so the two arms produce a controlled equal-work comparison.

5.2 Replay experiments and results

The 2×4 replay contains a broadcast plus two lowerings of the same 15 all-reduce events. All-reduce converges rank values in Routers and then uses a tree broadcast; it shares the prefix-reuse principle with multicast but is a distinct implementation. The configured bypass table selects ordinary XY for every trace path. Consequently, the Mesh Bypass arm is a noninterference control, while the 4.728× result below isolates request representation.

The broadcast replay expands 15 source events into 30 tree-multicast requests; both designs complete 6,720 source flits and 210 destination deliveries in a 17,047,500-tick window. Every path uses XY in this placement, giving a controlled equal-work bypass comparison.

Table 7 records the three replay views. Both topology arms are identical, confirming that unused physical links do not perturb the trace.

Table 7: Scaled H100 replay summary; p95 values use different logical units and must not be compared across scalar and tensor rows.

Replay view	Logical requests	Source flits	Window (ticks)	p95 (ticks)
Broadcast	30	6,720	17,047,500	643,500
Scalar all-reduce lanes	6,720	53,760	80,638,000	10,000
Tensor all-reduce	15	53,760	17,054,500	2,567,500

For all-reduce, both physical designs complete 15 logical tensor requests, 53,760 rank contributions, 53,760 exact lane reductions, 53,760 deliveries, and 147,840 Router flits. The tensor window is 17,054,500 ticks; p50/p95/p99 logical-request latencies are 647,500/2,567,500/2,567,500 ticks. These percentiles describe multi-flit requests and use a different request unit from the scalar-lane p95 of 10,000 ticks. Figure 9 visualizes the cross-lowering measurement window, where both views share the same network work.

Scalar lowering records an 80,638,000-tick window, so tensor streaming is 4.728× shorter on both Mesh XY and Mesh Bypass. Source and Router flit counts are identical, which isolates the measured gain to request representation and completion scheduling while rank contributions and network traffic remain fixed. The result characterizes the configured replay model; translating it to training-step speed requires an end-to-end study.

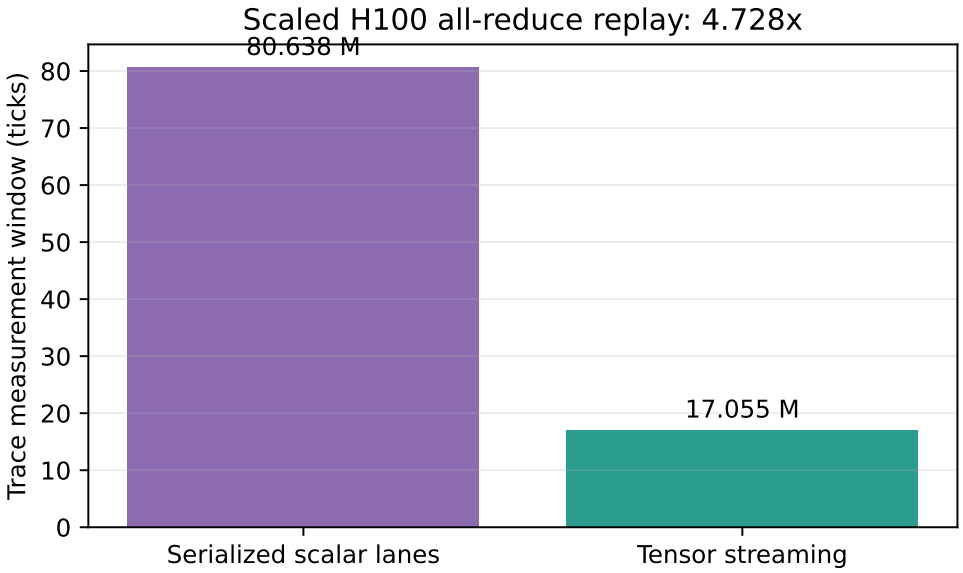


Figure 9: End-to-end replay window for scalar-lane and tensor-request lowerings of the same scaled all-reduce trace.

Trace-replay takeaway. Keeping a tensor as a streaming collective removes scalar serialization in this network model while preserving exact arithmetic and delivery invariants. The effect appears in the total trace window; per-request tail percentiles use different logical units for the two lowerings.

6 Limitations and interpretation boundary

The experiments establish the mechanisms within the tested model. The scope of the evidence is:

- The multicast matrix covers one 4×4 source placement (Router 5); corner, edge, and center source placements are future coverage. Its dedicated fast path and atomic fanout expose branch serialization under load.
- Bypass wire cost uses a Garnet latency, serialization, and static-cost proxy. Area, power, repeaters, and timing closure remain outside the model. Refined stride routes may use multiple monotonic express hops; their complete channel-dependency graph is audited before simulation.
- The bypass screen uses synthetic unicast traffic on data VNet 2. The distance-scaled screen is the hardware-oriented comparison, with optimistic timing serving as a sensitivity point.
- The H100 input is an executed collective microbenchmark with 1/1024 byte and time scaling. Full model traces are future workload coverage. Broadcast and tensor replay on the shared topology use ordinary XY paths.
- Tensor accumulation uses software state with arithmetic latency and finite table capacity outside the model. Chunk boundaries are validated by the contract and carried by the replay protocol.
- The results characterize cycle-level Garnet mechanisms. NVLink/NVSwitch measurements and end-to-end model throughput are future validation targets.

7 Division of labor

Chunyu Liu authored multicast and bypass implementation, correctness checks, trace capture, optimization, and the associated analysis. Boyan Pu authored the tensor all-reduce datapath, replay contract, validator, backpressure stress cases, and tensor statistics. Both authors integrated branches, reviewed regressions, reran the final quantitative snapshot, and verified this report.

8 Reproducibility and artifact trail

Compact accepted snapshots are in `report/data/`; generated vector and preview figures are in `report/figures/`; raw outputs are git-ignored under `report/raw-results/head-77fcf26d/`. The source trace and replay hashes, software versions, scaling semantics, and artifact roots are recorded in `report/data/trace_replay_provenance.txt`. The compact-data builder rejects wrong run counts, non-finite values, failed regression stages, inconsistent VNet or routing metadata, incomplete multicast pairs, and missing designs before writing summary tables or figures.

The full command matrix is retained in `report/README.md`. The compact snapshot and every figure can be rebuilt with:

```
python3 tests/gem5/lab4/run_lab4_full_gate.py --jobs 64
python3 report/scripts/build_final_report_data.py
python3 report/scripts/plot_multicast.py
python3 report/scripts/generate_architecture_figures.py
```

The trace-replay runners retain their original four-link diagonal topology as a controlled H100 case-study arm. The new multi-hop stride result belongs to the broad synthetic screen and is not retroactively substituted into that fixed trace placement.

9 Conclusion

The project evaluates three complementary mechanisms with three matching forms of evidence. Tree multicast shares duplicate prefixes and removes 39.51% of internal-link flits on average, with the saving rising to 53.13% at fanout 16; atomic fanout shapes the workload-dependent throughput curve. Express-link gains depend on topology, wire model, and load: the source-only distance-scaled diagonal route table reaches 1.131× geometric-mean latency speedup, while source-only stride links are neutral at 0.999×. The cycle-aware multi-hop stride refinement reaches 1.430× with 21.87% higher throughput and only one regression larger than 5%. Tensor all-reduce preserves the collective as 15 multi-flit requests and shortens the scaled H100 replay window by 4.728× while rank contributions and Router flits remain fixed.

Together these results support a narrow but useful design principle for collective-aware NoCs: expose structure where it removes demonstrable network work, admit physical shortcuts only after accounting for wire cost and the target traffic regime, and evaluate request-level streaming with a completion contract that makes its units explicit.

References

[1] N. Binkert et al., "The gem5 Simulator," *ACM SIGARCH Computer Architecture News*, vol. 39, no. 2, 2011, pp. 1–7. doi: 10.1145/2024716.2024718.

[2] N. Agarwal, T. Krishna, L.-S. Peh, and N. K. Jha, "GARNET: A Detailed On-Chip Network Model inside a Full-System Simulator," *ISPASS*, 2009, pp. 33–42. doi: 10.1109/ISPASS.2009.4919636.

[3] NVIDIA, "NVIDIA Collective Communication Library Documentation." https://docs.nvidia.com/deeplearning/nccl/user-guide/.